

Critic of Code

We approached the task by analysing Alex's unfinished code, identifying the gaps and incomplete sections. Modular functions for feature transformation, `toFeatureVector`, `similarity`, `loadData`, `KNNClassify` are already present. However the code currently only compares features of id and genre, and doesn't use any other comparators, these have now been implemented, by using a `normalize` function we ensure each piece of data is added to the feature after the average of all that type of feature has been subtracted. Alex's code also lacked any vector operations which we have added in full.

Additionally `KNNClassifier` function needs to be modified to support any integer value of k rather than being restricted to $k=1$. Another small issue in Alex's code is detailed in Figure 3, where a comment causes confusion about the parsing of features from the csv file. To improve Alex's initial implementation, we extended the `KNNClassify` function to support an arbitrary integer value of k . We have included a breakdown of our KNN Classify code at the bottom of the document.

Alex used `movieID` as a feature, this is a unique identifier for each movie and has no part in any attributes which makes the movie enjoyable or not. By using this as a feature Alex has risked introducing noise into his model, which could lead it to find patterns in `movieID` which don't correlate to anything meaningful. To fix this we have introduced a weight of zero for `movieID`, which nullifies it. By Alex purely using genre and `movieID` as his only features, his outcome accuracy is going to be incredibly inaccurate as these are not fully responsible for whether the movie will be liked. By adding in additional features this will broaden the end result and improve accuracy. The correlation between each feature and the outcome of Like It has been shown in Figure 4. The distribution of features within the data files plays a significant role in the outcome accuracy, as some features have very erratic distribution of values, shown in Figures 5 and 6. In our implementation, we introduced a weight vector that assigns specific weights to each feature based on its relevance to the outcome of "Like It". By introducing normalization we have ensured all features are treated equally, however this shouldn't be the case as some features have more predictive power than others and so require higher weighting.

The dot product in Alex's model has been removed and replaced with the Euclidean Distance, which measures the straight line distance between two points. This shows similarity between 2 points if they have a smaller distance than compared to other points, it takes into account both magnitude and direction of feature values. However the dot product does not directly measure similarity, and instead measures the degree of alignment between points. The dot product can be affected by the magnitude of vectors, as larger magnitude vectors naturally produce larger dot products even if the vectors are not aligned, and so Euclidean Distance gives a more accurate view of similarity between points for our mode shown in figure 2.

Within Alex's code, he has attempted to use one hot encoding to add genre as a feature, however has failed to do this, and instead has changed `feature[1]` to a number based on each genre, meaning, for example, "Action" has been weighted as 0 and "Thriller" has been weighted to 7, with this current implementation, genre becomes weighted differently in the features based on the type of genre. This original implementation has been shown in Figure

1 along with the new correct implementation of one hot encoding for genre, where each genre is assigned to its own feature position and only set to 1, when that genre movie is present.

As already discussed we have changed Alex's code to increase his accuracy output for predicting the likability of Movies. Firstly we added in the rest of the features to give the model all the information it needs to make an accurate prediction based on the full data set. From these features we then added weights to each of them, as already discussed, these weights were based on the correlation between features and like it, shown in Figure 4, and were adjusted suitably based on the outcome accuracy. Finally by completing `KNNClassify` for any value of k , this allowed us to select a k value of any integer, and select the value that gave us the highest accuracy. Our end result was an accuracy of 67% within our KNN model, by using a value of k being 1 or 3, both yielding the same accuracy.

When comparing KNN and Naive Bayes we see how the algorithms treat the data and their performances as shown in figures 4,5 and 6 of the report. Starting with KNN it is flexible because of its proximity based classification which makes it robust to different data distributions as seen in figure 6. This shows unpredictable distributions across numerical features. KNN handles this kind of inconsistency by normalising and weighting features with a final accuracy of 67% as stated in the report. However naive bayes on the other hand computes posterior probabilities assuming independence features which is a gross simplification of the relationship within a dataset. For example figure 4s heat map shows strong correlations among features like IMDB and Box Office Revenue that naive bayes cannot take full advantage of because of its independence assumption. While naive bayes is very responsive with an accuracy of 63% shown in the report its Gaussian probability distribution can be a poor representation for features that are not normally distributed such as the ones in figure 6. In figure 5 the genre distribution gives further evidence of KNNs capability to make effective use of one hot coded categorical feature. Continuing on naive bayes can struggle with features due to potential overlaps in the probability space. Naive bayes is efficient and interpretable, it is the adaptability of KNN that offers superior predictive power in this context as demonstrated by the reports results.

Figure 1

Alex's implementation of one hot

```
switch (genre) { // We also use represent each genre as a one-hot vector
    case "Action": feature[1] = 0; break;
    case "Fantasy": feature[1] = 1; break;
    case "Romance": feature[1] = 2; break;
    case "Sci-Fi": feature[1] = 3; break;
    case "Adventure": feature[1] = 4; break;
    case "Horror": feature[1] = 5; break;
    case "Comedy": feature[1] = 6; break;
    case "Thriller": feature[1] = 7; break;
    default: Assert(res: false);
}
```

Correct New Implementation

```
switch (genre) {
    case "Action": feature[1] = 1; break;
    case "Fantasy": feature[2] = 1; break;
    case "Romance": feature[3] = 1; break;
    case "Sci-Fi": feature[4] = 1; break;
    case "Adventure": feature[5] = 1; break;
    case "Horror": feature[6] = 1; break;
    case "Comedy": feature[7] = 1; break;
    case "Thriller": feature[8] = 1; break;
    default: Assert(res: false);
}
```

Figure 2

To check the similarity, a new method needs to be returned which contains the dot product operation, by using the euclidean distance.

```
static double similarity(double[] u, double[] v) {
    return dot(u, v);
}
```

The new method is:

```
static double similarity(double[] u, double[] v) { 1 usage new *
    return weightedEuclideanDistance(u, v);
}
static double weightedEuclideanDistance(double[] u, double[] v)
    double sum = 0;
    for (int i = 0; i < u.length; i++) {
        double diff = u[i] - v[i];
        sum += featureWeights[i] * Math.pow(diff, 2);
    }
    return Math.sqrt(sum);
}
```

Figure 3, Load Data Incorrect Comment Line

```
// Assuming csv format: MovieID,Title,Genre,Runtime,Year,Lead
Actor,Director,IMDB,RT(%),Budget,Box Office Revenue (in million $),Like it
double id = Double.parseDouble(values[0]);
String genre = values[2];
double runtime = Double.parseDouble(values[10]);
double year = Double.parseDouble(values[3]);
double imdb = Double.parseDouble(values[7]);
double rt = Double.parseDouble(values[6]);
double budget = Double.parseDouble(values[9]);
double boxOffice = Double.parseDouble(values[8]);
```

Within Alex’s loadData function, he states an Assumption of the csv format as, “MovieID, Title, Genre, Runtime, Year, Lead Actor, Director,IMDB, RT(%), Budget, Box Office Revenue (in million \$), Like it”, where the actual csv file is, “MovieID, Title, Genre, Year, Director, Lead Actor, RT (% ,IMDB, Box Office Revenue (in million \$), Budget, Runtime, Like it”. The parsing of the data from the file is correct however so this does not cause any error in the program and will only cause confusion for whoever is examining the code.

Figure 4 Correlation of Features Heatmap

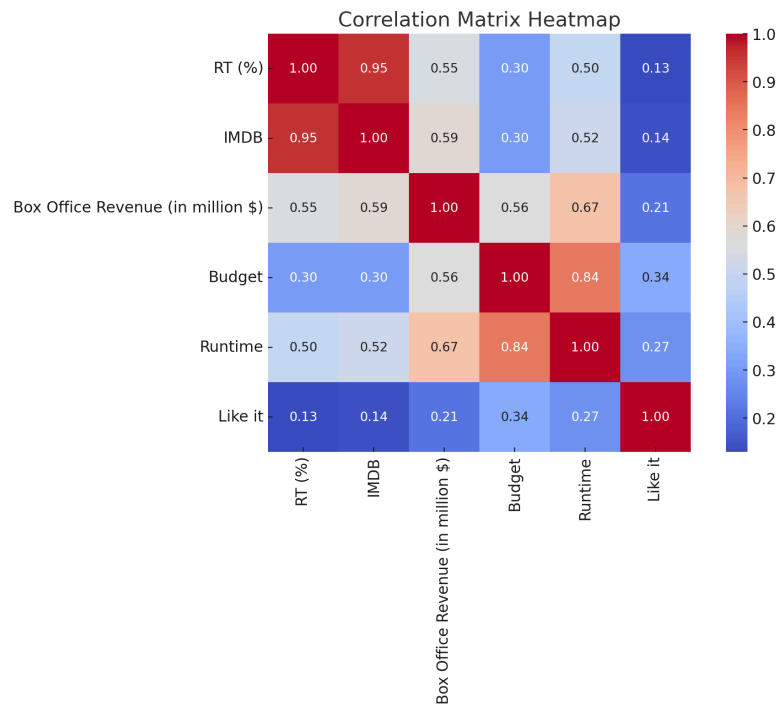


Figure 5 Distribution of Movies By Genre

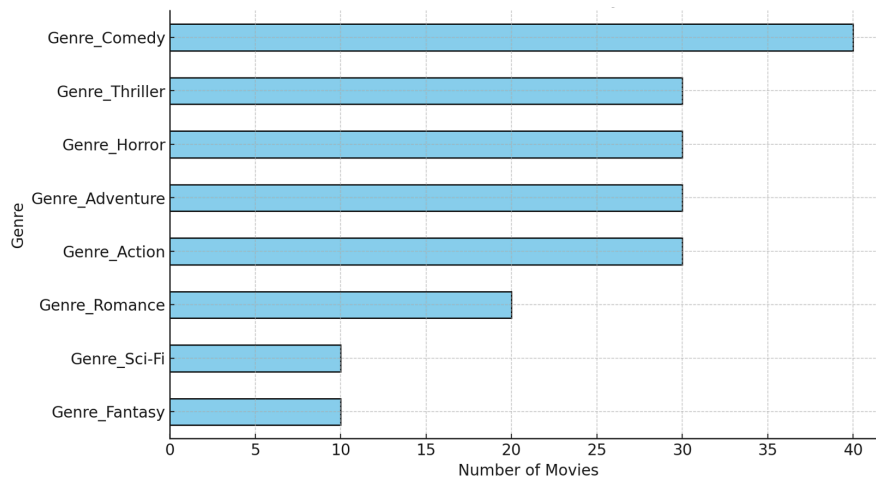
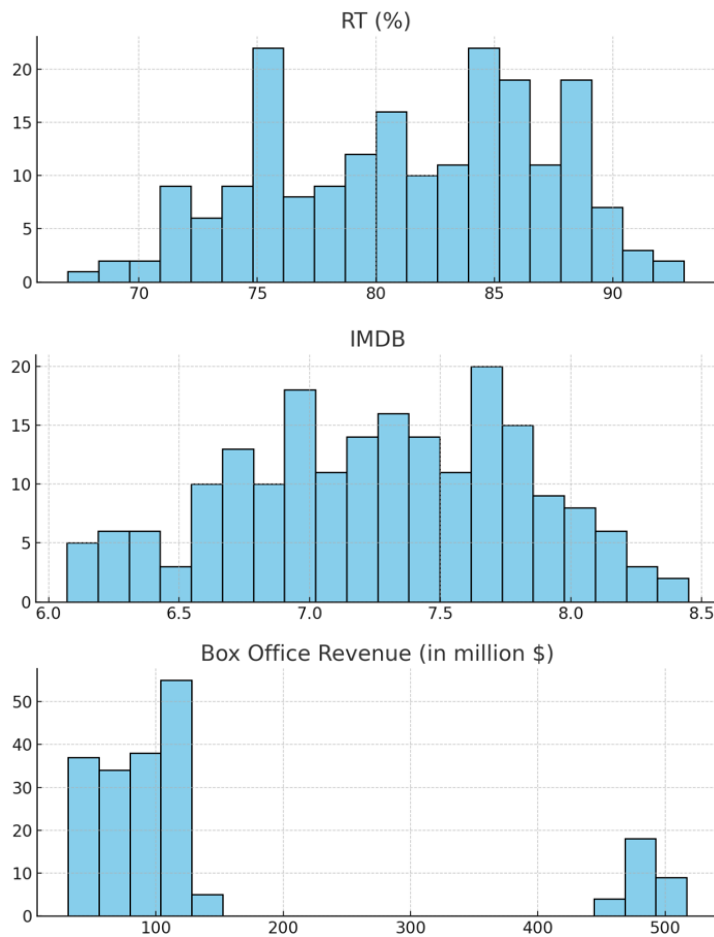
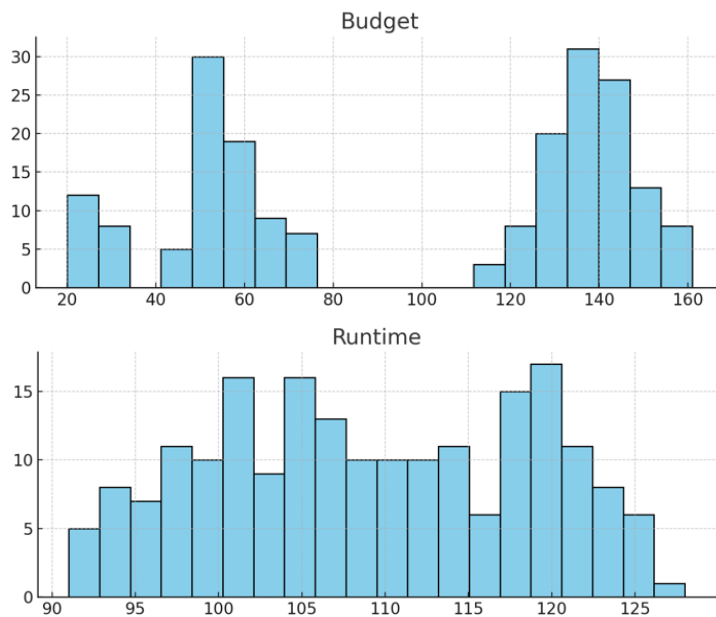


Figure 6 Distribution of Numerical Features





KNNClassify Breakdown of Code

It starts by initializing two arrays: `nearestLabels` to store the labels of the K nearest neighbors and `nearestDistances` to hold their similarity scores. The distances are initially filled with the smallest possible value in java to ensure they can be replaced with actual similarities. The function then iterates through the training data, calculating the similarity between each training point and the test feature using the similarity function. If the similarity of a training point is greater than one of the current K nearest neighbors, the function shifts less similar neighbors down the list to maintain sorted order, inserts the new point into the appropriate position, and updates the label and distance arrays accordingly. Once all training points have been evaluated, the function counts the occurrences of each label among the K nearest neighbors using an array, assuming labels range from 0 to 9. Finally, it identifies the label with the highest count (the majority label) and returns it as the predicted class for the test feature.

Naïve Bayes Explanation

It calculates the posterior probabilities for which Alex likes the movies or not “1” or “0”. It starts with calculating the variance and the mean in each feature which are calculated independently for every class. These show how the feature values for each class are distributed. Then, prior probability needs to be calculated where The fraction of training samples that correspond to a particular class is used to compute the prior probability, and then stored in the Priors array and all that calculation is done in calculateMeanVariance. Moreover, in predictNaiveBayes we use the Gaussian normal distribution formula which is used to calculate the likelihood of a feature value given the class, assuming that features are normally distributed, The formula is:

$$f(x; \mu, \sigma^2) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

Here μ is the average,

σ is the standard deviation

f is the probability density

π is pi

In addition, Calculate the prior and likelihood probability combined and then stored in posteriors. It calculates as follows: *posterior probability = Prior Probability * likelihood*
Lastly, the prediction would be taken from the class with the highest posterior probability. The end result of The Naive Bayes model within our program is an accuracy of 63%